Improving PPO in Sparse-Reward MiniGrid with LLM-Suggested Subgoals as Intrinsic Rewards

Ole
github.com/Ole-rgb/TEMPPO

1 Motivation

I want to build an improved PPO for sparse-reward tasks: PPO extended with an intrinsic reward for achieving LLM-suggested subgoals, with the explicit aim of beating vanilla PPO *and* the standard exploration bonuses (RND, count-based) on hard MiniGrid environments. The approach is inspired by ELLM (1), but differs in three ways: (1) ELLM uses LLM goals for *reward-free pretraining* of a goal-conditioned agent, whereas I add the subgoal bonus directly to the task reward to solve the target task online; (2) ELLM matches achieved behavior to goals via embedding similarity with a tuned threshold, whereas I constrain the LLM to a closed template vocabulary and match via exact predicates; (3) LLM calls are episodic and cached (no per-step foundation-model queries). Because the extension is pure reward relabeling, it can also be stacked on top of existing bonuses. I therefore additionally test whether LLM subgoals and RND are complementary (directed semantic rewards plus undirected novelty rewards) as suggested by results in (3).

2 Related Topics

This builds on the exploration lecture (intrinsic rewards, count-based exploration, novelty prediction). Literature: ELLM (1) introduced LLM-subgoal intrinsic rewards (embedding matching, compared against RND); DLLM (2) rewards LLM subgoal hints inside world-model rollouts and VSIMR+LLM (3) combines LLM intrinsic rewards with VAE novelty on MiniGrid DoorKey (A2C). Semantics-free baselines: RND (4) and count-based exploration (5). Implementations build on CleanRL (6) and MiniGrid (7).

3 Idea

A hand-written captioner decodes MiniGrid's $7\times7\times3$ observation into text. Once per episode, the LLM receives caption + mission and returns subgoals *constrained to a closed template vocabulary* ("pick up the ⟨color⟩ ⟨object⟩", "open the ⟨color⟩ door", ...). A local, LLM-free matcher checks transitions against suggested goals, and rewards the first achievement of a suggested goal per episode λ (Eq. 1). Captions repeat, so LLM responses are cached. The full experiment runs with a local Llama-8B.

$$r_t = r_t^{\text{env}} + \lambda \cdot \mathbb{1}\left[\text{transition } t \text{ achieves a suggested, not-yet-achieved subgoal}\right] \tag{1}$$

I compare this extension to RND (and count-based bonuses) tuned with the same budget as the LLM variant, so no method wins by tuning alone. Since both extensions are additive reward bonuses, they compose without interacting: I additionally evaluate the combination

$$r_t = r_t^{\text{env}} + \lambda_{\text{LLM}} \cdot \mathbb{1}\left[\text{subgoal achieved}\right] + \lambda_{\text{RND}} \cdot \text{novelty}(s_{t+1}), \tag{2}$$

where each λ is tuned in isolation and reused in the combination (joint tuning would exceed the compute budget).

Algorithm 1 PPO with LLM-suggested subgoal rewards (per episode).

Require: environment e, agent A, goal source L (LLM / none), captioner c, matcher m, λ, cache C
  return policy π
  At episode start: $g \leftarrow C[c(s_0)]$ or $g \leftarrow L(c(s_0))$; store in C
  for each step t **do**
    $r_t \leftarrow r_t^{\text{env}} + \lambda \cdot m(s_t, a_t, s_{t+1}, g)$ ▷ exact predicate match, first achievement only
  end for
  Update A with PPO

4 Experiments

Environments & Metrics Three sparse-reward MiniGrid environments of increasing difficulty: DoorKey-8x8, MultiRoom-N4-S5, KeyCorridorS3R2. Metrics: evaluation return (IQM with stratified bootstrap CIs), steps to solve-threshold (, state coverage, fraction of suggested subgoals achieved).

Experimental Scope Five conditions per environment: (1) vanilla PPO, (2) PPO + count-based bonus, (3) PPO + RND, (4) PPO + LLM subgoals, (5) PPO + RND + LLM subgoals (scales reused from the isolated tuning of (3) and (4)). Matched tuning: each bonus method gets the same small grid over its bonus scale (3 values $\times$ 3 seeds). Main comparison at 10 seeds per condition per environment ($5 \times 3 \times 10 = 150$ runs plus $\sim$80 tuning runs).

Estimated Computational Load MiniGrid + PPO with a small CNN trains in roughly 10-15 minutes per run (1M steps, laptop CPU or Colab GPU). Total $\approx$ 230 runs $\times$ $\sim$10 min $\approx$ 40 compute-hours. Cached, template-constrained LLM usage totals a few hundred short calls across all experiments.

5 Timeline

Total: roughly two weeks of full-time effort.

- Setup (2 days): project and repository setup, adapting CleanRL's PPO from Atari to MiniGrid, prompt design.
- Implementation (4 days): captioner, template parser, exact-predicate matcher, cached LLM interface, random-goal control, RND/count-based baselines on a shared CleanRL PPO codebase. (Captioner, matcher and wrapper are already prototyped and tested.)
- Experiments (4 days): matched tuning grids, main comparison, ablations; continuous monitoring.
- Analysis (1 days): IQM/CI aggregation, coverage visualizations.
- Reporting (3 days): report, poster, repository cleanup (incl. committed LLM cache) and reproducibility checklist.

References

[1] Du et al. *Guiding Pretraining in Reinforcement Learning with Large Language Models*. ICML, 2023.

[2] Ma et al. *World Models with Hints of Large Language Models for Goal Achieving*. arXiv:2406.07381, 2024.

[3] *LLM-Driven Intrinsic Motivation for Sparse Reward Reinforcement Learning*. arXiv:2508.18420, 2025.

[4] Burda et al. *Exploration by Random Network Distillation*. ICLR, 2019.

[5] Bellemare et al. *Unifying Count-Based Exploration and Intrinsic Motivation*. NeurIPS, 2016.

[6] Huang et al. *CleanRL: High-quality Single-file Implementations of Deep RL Algorithms*. JMLR, 2022.

[7] Chevalier-Boisvert et al. *Minigrid & Miniworld: Modular and Customizable RL Environments*. NeurIPS Datasets and Benchmarks, 2023.